

人工智能与机器学习

2023年10月

3.13 信息论

2

- 信息论是应用数学的一个分支，主要研究的是对一个信号包含信息的多少进行量化。
- 它最初被发明是用来研究在一个含有噪声的信道上用离散的字母表来发送消息，例如通过无线电传输来通信。在这种情况下，信息论告诉我们如何对使用多种不同编码机制、从特定的概率分布上采样得到的消息，设计最优编码以及计算消息的期望长度
- 消息中出现概率较大的字母使用较短的编码长度，而出现概率较小的字母，则使用较长的编码长度

3.13 信息论

3

- 在通信系统中，发送方通过编码将信息转换为数字信号，接收方则通过解码将数字信号转换为原始信息。
- 在机器学习中，我们也可以把信息论应用于连续型变量，此时某些消息长度的解释不再适用。
- 信息论是电子工程和计算机科学中许多领域的基础。

描述概率分布

量化概率分布之间的相似性

3.13 信息论

4

- 信息论的基本想法是：
 - 一个不太可能的事件居然发生了，要比一个非常可能的事件发生，能提供更多的信息。
 - 消息说：“今天早上太阳升起”信息量是如此之少以至于没有必要发送，但一条消息说：“今天早上有日食”信息量就很丰富。

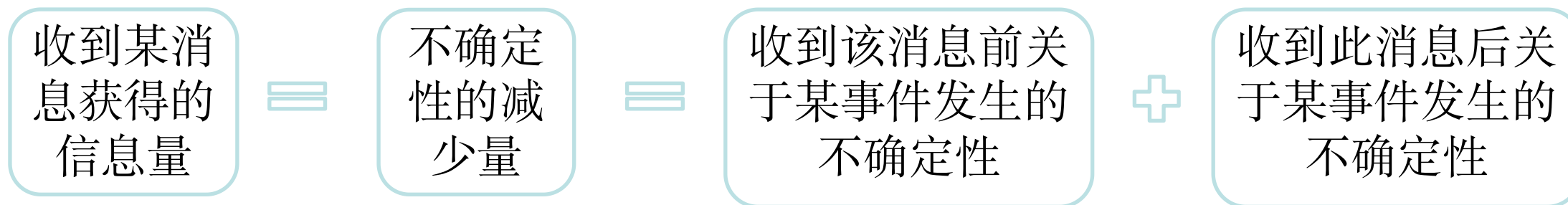
如何去量化信息？

3.13 信息论

5

如何考量化信息？

- 信源的信息实质：不确定性
 - 信源输出的是消息，消息的内涵是信息。信源输出一个符号，我们认为发生一个事件。
 - 信息量的大小与不确定性的消除多少有关



3.13 信息论

6

- 非常可能发生的事件信息量要比较少
 - 极端情况下，确保一定发生的事件应该没有信息量。
- 较不可能发生的事件具有更高的信息量
- 独立事件应具有增量的信息
 - 例如，投掷的硬币两次正面朝上传递的信息量，应该是投掷一次硬币正面朝上的信息量的两倍。

3.13 信息论

7

- 为了满足上述三个性质，我们定义一个事件 $X = x$ 的 **自信息** (self-information) 为：

$$I(x) = -\log P(x).$$

- 该式是用 $\log$ 来表示自然对数，其底数为 e
- $I(x)$ 单位是**奈特 (nats)**。一奈特是**以 e 分之一的概率观测到一个事件时获得的信息量**。
- 使用底数为 2 的对数，单位是**比特 (bit)** 或者香农 (shannons)；通过比特度量的信息只是通过奈特度量信息的常数倍。

3.13 信息论

8

- 总结来说, 在事件发生前, 自信息表示事件发生的不确定性。
- 在事件发生后, 自信息表示事件所包含的信息量, 是提供给信宿的信息量, 也是解除这种不确定性所需要的信息量

$$I(x) = -\log_2 p(x)$$

假设某个信源以概率 $p=0.25$ 发出符号A, 则A的自信息= ?

若某信源以概率 $p=0.01$ 发出符号B, 则B的自信息= ?

若某信源以概率 $p=0.99$ 发出符号C, 则C的自信息= ?

3.13 信息论

9

自信息只处理单个的输出。我们可以用 香农熵 (Shannon entropy) 来对**整个概率分布中的不确定性总量**进行量化:

$$H(x) = \mathbb{E}_{x \sim P}[I(x)] = -\mathbb{E}_{x \sim P}[\log P(x)],$$

也记作 $H(P)$ 。换言之, 一个分布的香农熵是指遵循这个分布的事件所产生的期望信息总量。它给出了对依据概率分布 P 生成的符号进行编码所需的比特数在平均意义上的下界

3.13 信息论

10

- 那些接近确定性的分布 (输出几乎可以确定) 具有较低的熵; 那些接近均匀分布的概率分布具有较高的熵。当 x 是连续的, 香农熵被称为 微分熵

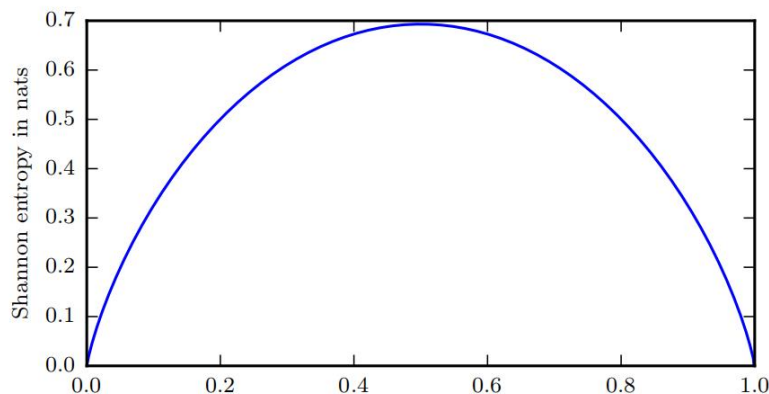


图 3.5: 二值随机变量的香农熵。该图说明了更接近确定性的分布是如何具有较低的香农熵, 而更接近均匀分布的分布是如何具有较高的香农熵。水平轴是 p , 表示二值随机变量等于 1 的概率。熵由 $(p-1)\log(1-p) - p\log p$ 给出。当 p 接近 0 时, 分布几乎是确定的, 因为随机变量几乎总是 0。当 p 接近 1 时, 分布也几乎是确定的, 因为随机变量几乎总是 1。当 $p = 0.5$ 时, 熵是最大的, 因为分布在两个结果 (0 和 1) 上是均匀的。

3.13 信息论

11

如果我们对于同一个随机变量 x 有两个单独的概率分布 $P(x)$ 和 $Q(x)$ ，我们可以使用 **KL 散度**（Kullback-Leibler (KL) divergence）来衡量这两个分布的差异：

$$D_{\text{KL}}(P||Q) = \mathbb{E}_{x \sim P} \left[\log \frac{P(x)}{Q(x)} \right] = \mathbb{E}_{x \sim P} [\log P(x) - \log Q(x)]. \quad (3.50)$$

3.13 信息论

12

$$D_{\text{KL}}(P||Q) = \mathbb{E}_{x \sim P} \left[\log \frac{P(x)}{Q(x)} \right] = \mathbb{E}_{x \sim P} [\log P(x) - \log Q(x)].$$

- 在离散型变量的情况下，KL 散度衡量的是，当我们使用一种被设计成能够使得概率分布 **Q** 产生的消息的长度最小的编码，发送包含由概率分布 **P** 产生的符号的消息时，所需要的**额外信息量**。

3.13 信息论

13

- KL 散度有很多有用的性质，最重要的是它是非负的。
- 因为 KL 散度是非负的并且衡量的是两个分布之间的差异，它经常被用作分布之间的某种距离。
- 然而，它并不是真的距离因为它不是对称的

$$D_{\text{KL}}(P||Q) \neq D_{\text{KL}}(Q||P)$$

3.13 信息论

14

一个和 KL 散度密切联系的量是 **交叉熵** (cross-entropy) $H(P, Q) = H(P) + D_{\text{KL}}(P||Q)$, 它和 KL 散度很像但是缺少左边一项:

$$H(P, Q) = -\mathbb{E}_{x \sim P} \log Q(x). \quad (3.51)$$

针对 Q 最小化交叉熵等价于最小化 KL 散度, 因为 Q 并不参与被省略的那一项。

3.13 结构化概率模型

15

机器学习的算法经常会涉及到在非常多的随机变量上的概率分布。通常，这些概率分布涉及到的直接相互作用都是介于非常少的变量之间的。使用单个函数来描述整个联合概率分布是非常低效的（无论是计算上还是统计上）。

我们可以把概率分布分解成许多因子的乘积形式，而不是使用单一的函数来表示概率分布。例如，假设我们有三个随机变量 a , b 和 c ，并且 a 影响 b 的取值， b 影响 c 的取值，但是 a 和 c 在给定 b 时是条件独立的。我们可以把全部三个变量的概率分布重新表示为两个变量的概率分布的连乘形式：

$$p(a, b, c) = p(a)p(b | a)p(c | b). \quad (3.52)$$

3.13 结构化概率模型

16

我们可以用图来描述这种分解。这里我们使用的是图论中的“图”的概念：由一些可以通过边互相连接的顶点的集合构成。当我们用图来表示这种概率分布的分解，我们把它称为**结构化概率模型**（structured probabilistic model）或者**图模型**（graphical model）。

有两种主要的结构化概率模型：有向的和无向的。两种图模型都使用图 $\mathcal{G}$ ，其中图的每个节点对应着一个随机变量，连接两个随机变量的边意味着概率分布可以表示成这两个随机变量之间的直接作用。

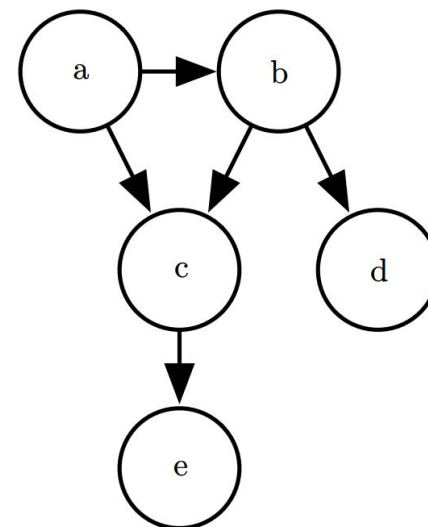
3.13 结构化概率模型

17

有向（directed）模型使用带有有向边的图，它们用条件概率分布来表示分解，就像上面的例子。特别地，有向模型对于分布中的每一个随机变量 x_i 都包含着一个影响因子，这个组成 x_i 条件概率的影响因子被称为 x_i 的父节点，记为 $Pa_G(x_i)$ ：

$$p(\mathbf{x}) = \prod_i p(x_i \mid Pa_G(x_i)). \quad (3.53)$$

图 3.7 给出了一个有向图的例子以及它表示的概率分布的分解。



$$p(a, b, c, d, e) = \quad ?$$

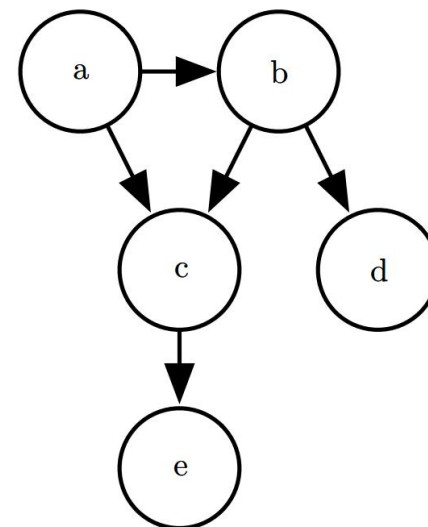
3.13 结构化概率模型

18

有向（directed）模型使用带有有向边的图，它们用条件概率分布来表示分解，就像上面的例子。特别地，有向模型对于分布中的每一个随机变量 x_i 都包含着一个影响因子，这个组成 x_i 条件概率的影响因子被称为 x_i 的父节点，记为 $Pa_G(x_i)$ ：

$$p(\mathbf{x}) = \prod_i p(x_i \mid Pa_G(x_i)). \quad (3.53)$$

图 3.7 给出了一个有向图的例子以及它表示的概率分布的分解。



$$p(a, b, c, d, e) = p(a)p(b \mid a)p(c \mid a, b)p(d \mid b)p(e \mid c)$$

3.13 结构化概率模型

19

无向（undirected）模型使用带有无向边的图，它们将分解表示成一组函数；不像有向模型那样，这些函数通常不是任何类型的概率分布。 $\mathcal{G}$ 中任何满足两两之间有边连接的顶点的集合被称为团。无向模型中的每个团 $\mathcal{C}^{(i)}$ 都伴随着一个因子 $\phi^{(i)}(\mathcal{C}^{(i)})$ 。这些因子仅仅是函数，并不是概率分布。每个因子的输出都必须是非负的，但是并没有像概率分布中那样要求因子的和或者积分为 1。

3.13 结构化概率模型

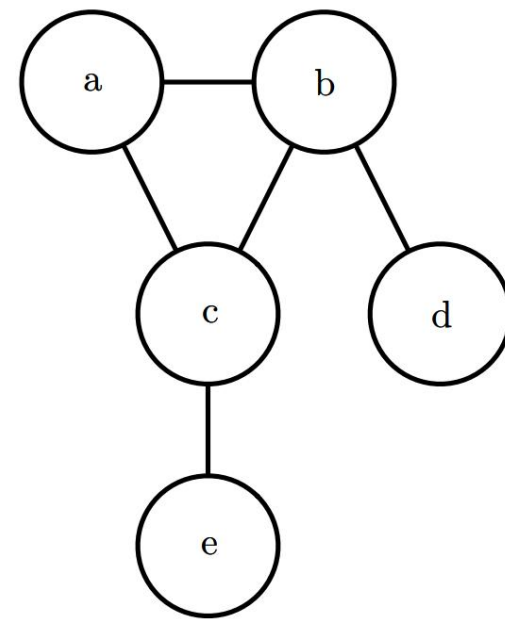
20

随机变量的联合概率与所有这些因子的乘积 **成比例**（proportional）——意味着因子的值越大则可能性越大。当然，不能保证这种乘积的求和为 1。所以我们需要除以一个归一化常数 Z 来得到归一化的概率分布，归一化常数 Z 被定义为 ϕ 函数乘积的所有状态的求和或积分。概率分布为：

$$p(\mathbf{x}) = \frac{1}{Z} \prod_i \phi^{(i)}(\mathcal{C}^{(i)}) . \quad (3.55)$$

图 3.8 给出了一个无向图的例子以及它表示的概率分布的分解。

$$p(a, b, c, d, e) = \frac{1}{Z} \phi^{(1)}(a, b, c) \phi^{(2)}(b, d) \phi^{(3)}(c, e)$$



第四章 数值计算

21

- 机器学习算法通常需要大量的数值计算。
 - 数值计算是指通过迭代过程更新解的估计值来解决数学问题的算法，而不是通过解析过程推导出公式来提供正确解的方法。
- 常见的操作包括优化（找到最小化或最大化函数值的参数）和线性方程组的求解。
- 对数字计算机来说实数无法在有限内存下精确表示，因此仅仅是计算涉及实数的函数也是困难的。

4.1 上溢和下溢

22

- 下溢

- 当接近零的数被四舍五入为零时发生下溢。
- 许多函数在其参数为零而不是一个很小的正数时才会表现出质的不同。例如，我们通常要避免被零除或避免取零的对数

- 上溢

- 当大量级的数被近似为 ∞ 或 $-\infty$ 时发生上溢。进一步的运算通常会导致这些无限值变为非数字。

4.1 上溢和下溢

23

- 连续数学在数字计算机上的根本困难是，我们需要通过有限数量的位模式来表示无限多的实数。这意味着我们在计算机中表示实数时，几乎总会引入一些近似误差。
- 在许多情况下，这仅仅是舍入误差。舍入误差会导致一些问题，特别是当许多操作复合时，即使是理论上可行的算法，如果在设计时没有考虑最小化舍入误差的累积，在实践时也可能会导致算法失效。

$$\text{softmax}(\mathbf{x})_i = \frac{\exp(x_i)}{\sum_{j=1}^n \exp(x_j)}$$

4.3 基于梯度的优化方法

24

大多数深度学习算法都涉及某种形式的优化。优化指的是改变 $\boldsymbol{x}$ 以最小化或最大化某个函数 $f(\boldsymbol{x})$ 的任务。我们通常以最小化 $f(\boldsymbol{x})$ 指代大多数最优化问题。最大化可经由最小化算法最小化 $-f(\boldsymbol{x})$ 来实现。

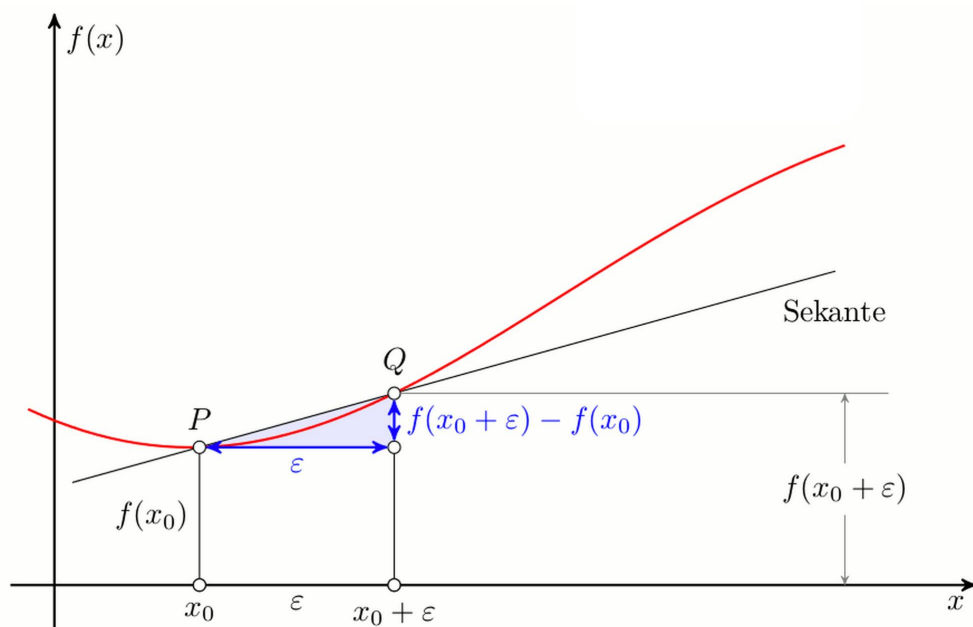
我们把要最小化或最大化的函数称为 **目标函数** (objective function) 或 **准则** (criterion)。当我们对其进行最小化时，我们也把它称为 **代价函数** (cost function)、**损失函数** (loss function) 或 **误差函数** (error function)。

$$\boldsymbol{x}^* = \arg \min f(\boldsymbol{x})$$

4.3 基于梯度的优化方法

25

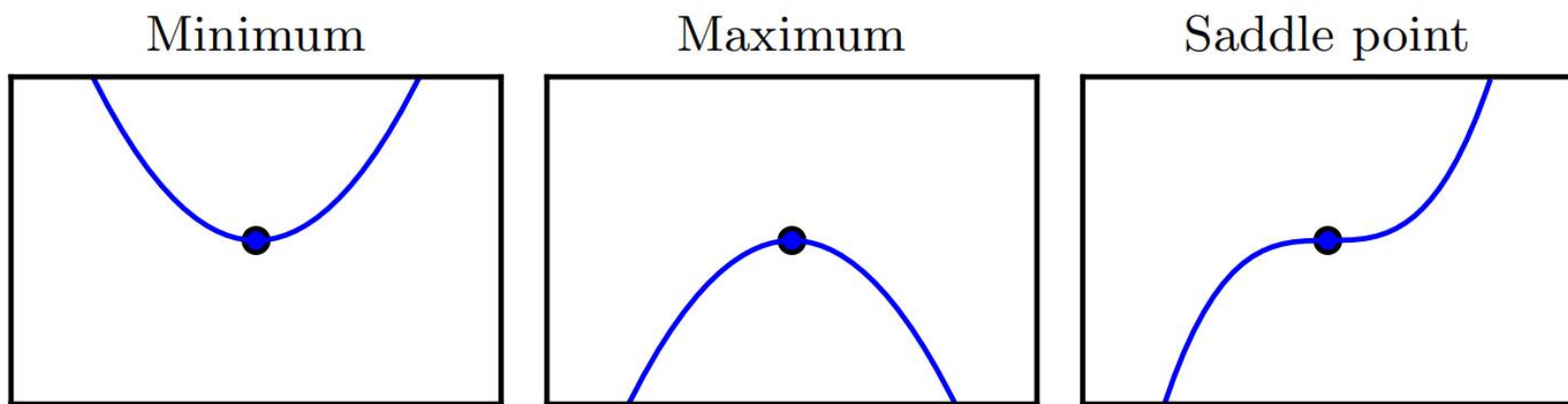
假设我们有一个函数 $y = f(x)$, 其中 x 和 y 是实数。这个函数的 **导数** (derivative) 记为 $f'(x)$ 或 $\frac{dy}{dx}$ 。导数 $f'(x)$ 代表 $f(x)$ 在点 x 处的斜率。换句话说, 它表明如何缩放输入的小变化才能在输出获得相应的变化: $f(x + \epsilon) \approx f(x) + \epsilon f'(x)$ 。



4.3 基于梯度的优化方法

26

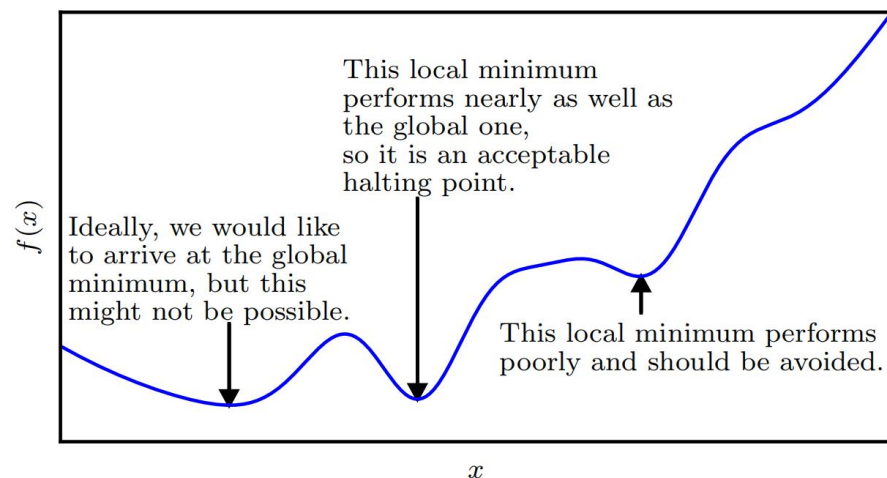
当 $f'(x) = 0$ ，导数无法提供往哪个方向移动的信息。 $f'(x) = 0$ 的点称为 **临界点**（critical point）或 **驻点**（stationary point）。一个 **局部极小点**（local minimum）意味着这个点的 $f(x)$ 小于所有邻近点，因此不可能通过移动无穷小的步长来减小 $f(x)$ 。一个 **局部极大点**（local maximum）意味着这个点的 $f(x)$ 大于所有邻近点，因此不可能通过移动无穷小的步长来增大 $f(x)$ 。有些临界点既不是最小点也不是最大点。这些点被称为 **鞍点**（saddle point）。



4.3 基于梯度的优化方法

27

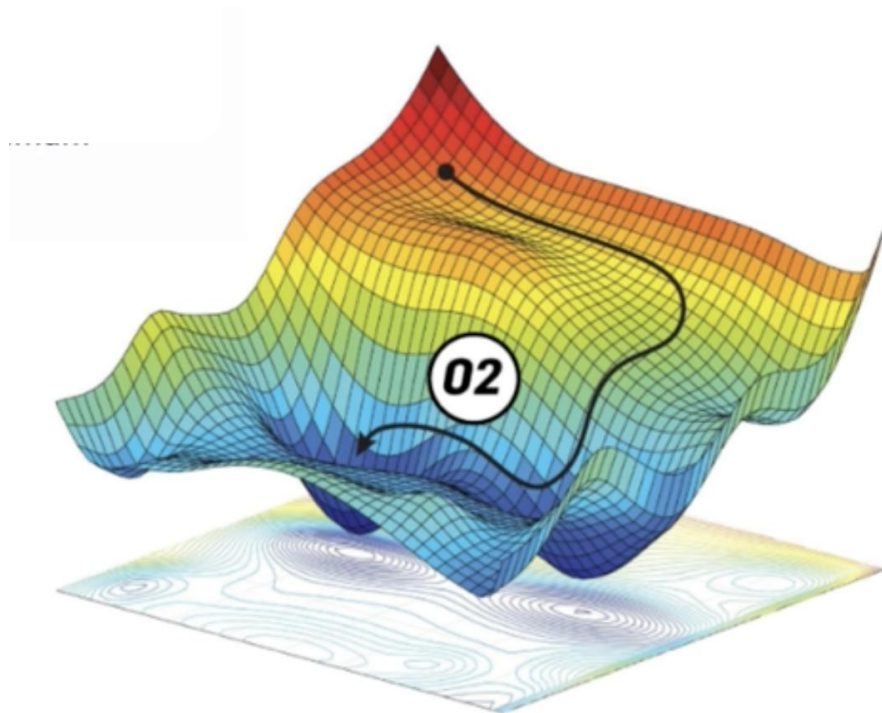
使 $f(x)$ 取得绝对的最小值（相对所有其他值）的点是**全局最小点**（global minimum）。函数可能只有一个全局最小点或存在多个全局最小点，还可能不存在不是全局最优的局部极小点。在深度学习的背景下，我们要优化的函数可能含有许多不是最优的局部极小点，或者还有很多处于非常平坦的区域内的鞍点。尤其是当输入是多维的时候，所有这些都使优化变得困难。因此，我们通常寻找使 f 非常小的点，但这在任何形式意义下并不一定是最小。



4.3 基于梯度的优化方法

28

针对具有多维输入的函数，我们需要用到 **偏导数**（partial derivative）的概念。
偏导数 $\frac{\partial}{\partial x_i} f(\mathbf{x})$ 衡量点 $\mathbf{x}$ 处只有 x_i 增加时 $f(\mathbf{x})$ 如何变化。



4.3 基于梯度的优化方法

29

在 $\mathbf{u}$ （单位向量）方向的**方向导数**（directional derivative）是函数 f 在 $\mathbf{u}$ 方向的斜率。换句话说，方向导数是函数 $f(\mathbf{x} + \alpha \mathbf{u})$ 关于 α 的导数（在 $\alpha = 0$ 时取得）。使用链式法则，我们可以看到当 $\alpha = 0$ 时， $\frac{\partial}{\partial \alpha} f(\mathbf{x} + \alpha \mathbf{u}) = \mathbf{u}^\top \nabla_{\mathbf{x}} f(\mathbf{x})$ 。

4.3 基于梯度的优化方法

30

对一个向量求导的导数: f 的导数是包含所有偏导数的向量, 记为 $\nabla_{\mathbf{x}}f(\mathbf{x})$ 。梯度的第 i 个元素是 f 关于 x_i 的偏导数。在多维情况下, 临界点是梯度中所有元素都为零的点。

梯度指示了函数的局部变化率最大的方向, 而方向导数则告诉我们, 当沿着该方向移动时, 函数的变化率是多少。

基于梯度的优化方法

31

- 导数
- 偏导数
- 方向导数
- 梯度

梯度指示了函数的局部变化率最大的方向，而方向导数则告诉我们，当沿着该方向移动时，函数的变化率是多少。

方向导数等于梯度与该方向的点积，方向导数最大的方向，为梯度方向

梯度方向指向“上坡”，负梯度方向指向“下坡”

4.3 基于梯度的优化方法

32

最速下降建议新的点为

减号


$$\boldsymbol{x}' = \boldsymbol{x} - \epsilon \nabla_{\boldsymbol{x}} f(\boldsymbol{x}) \quad (4.5)$$

其中 ϵ 为 **学习率** (learning rate), 是一个确定步长大小的正标量。我们可以通过几种不同的方式选择 ϵ 。普遍的方式是选择一个小常数。有时我们通过计算, 选择使方向导数消失的步长。还有一种方法是根据几个 ϵ 计算 $f(\boldsymbol{x} - \epsilon \nabla_{\boldsymbol{x}} f(\boldsymbol{x}))$, 并选择其中能产生最小目标函数值的 ϵ 。这种策略被称为线搜索。

学习率是梯度下降法的重要超参数

4.3 基于梯度的优化方法

33

函数:

$$J(\theta) = \theta^2$$

微分:

$$J'(\theta) = 2\theta$$

初始化, 起点:

$$\theta^0 = 1$$

学习率:

$$\alpha = 0.4$$

梯度下降公式

$$\Theta^1 = \Theta^0 - \alpha \nabla J(\Theta)$$

计算梯度下降四次后的位置, 并绘出梯度下降的迭代过程, 思考梯度下降的轨迹

4.3 基于梯度的优化方法

34

$$\theta^0 = 1$$

$$\theta^1 = \theta^0 - \alpha * J'(\theta^0)$$

$$= 1 - 0.4 * 2$$

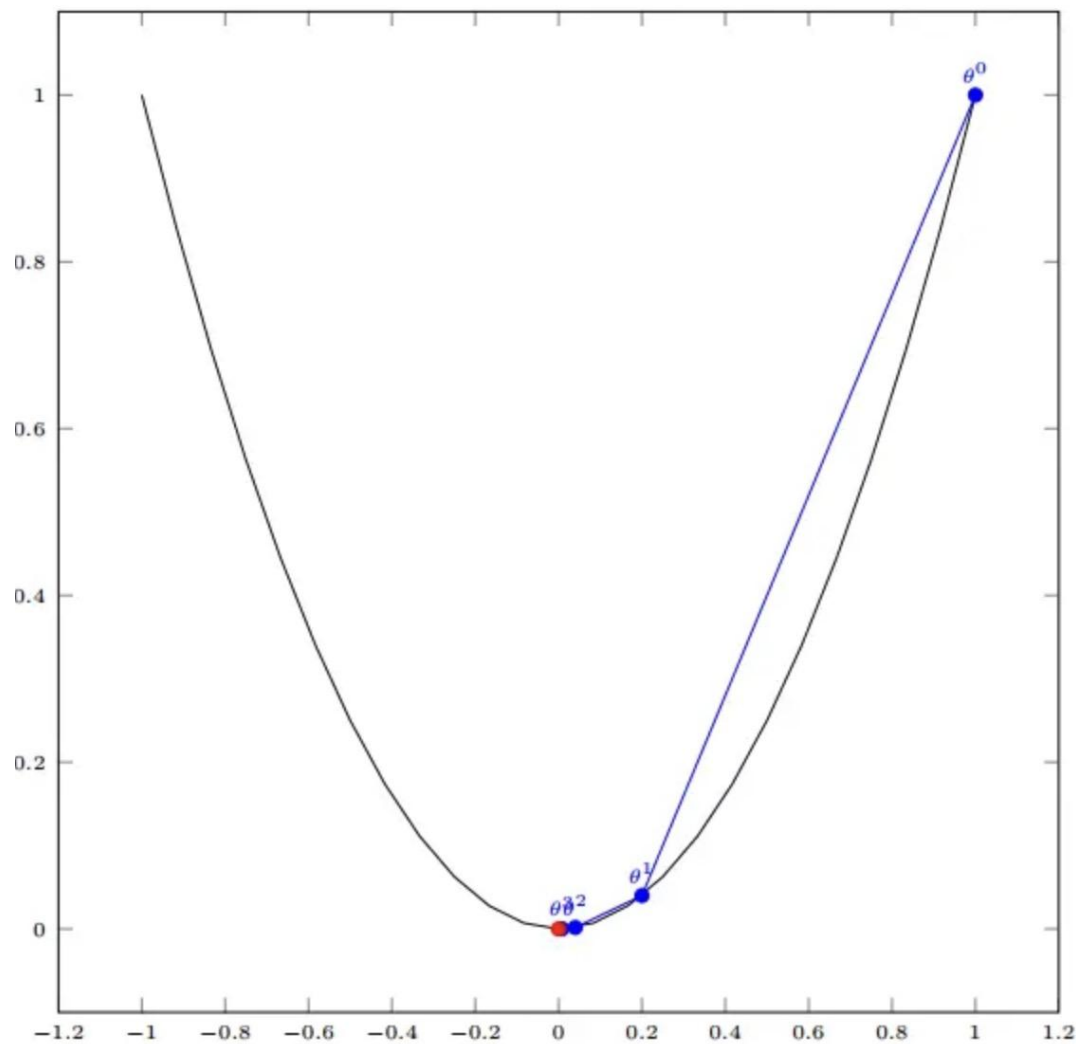
$$= 0.2$$

$$\theta^2 = \theta^1 - \alpha * J'(\theta^1)$$

$$= 0.04$$

$$\theta^3 = 0.008$$

$$\theta^4 = 0.0016$$



4.3 基于梯度的优化方法

35

函数:

$$J(\Theta) = \theta_1^2 + \theta_2^2$$

初始化, 起点:

$$\Theta^0 = (1, 3)$$

学习率:

$$\alpha = 0.1$$

梯度:

$$\nabla J(\Theta) = \langle 2\theta_1, 2\theta_2 \rangle$$

$$\Theta^0 = (1, 3)$$

$$\begin{aligned}\Theta^1 &= \Theta^0 - \alpha \nabla J(\Theta) \\ &= (1, 3) - 0.1(2, 6) \\ &= (0.8, 2.4)\end{aligned}$$

$$\begin{aligned}\Theta^2 &= (0.8, 2.4) - 0.1(1.6, 4.8) \\ &= (0.64, 1.92)\end{aligned}$$

$$\Theta^3 = (0.512, 1.536)$$

$$\Theta^4 = (0.4096, 1.2288000000000001)$$

$\vdots$

$$\Theta^{10} = (0.10737418240000003, 0.32212254720000005)$$

$\vdots$

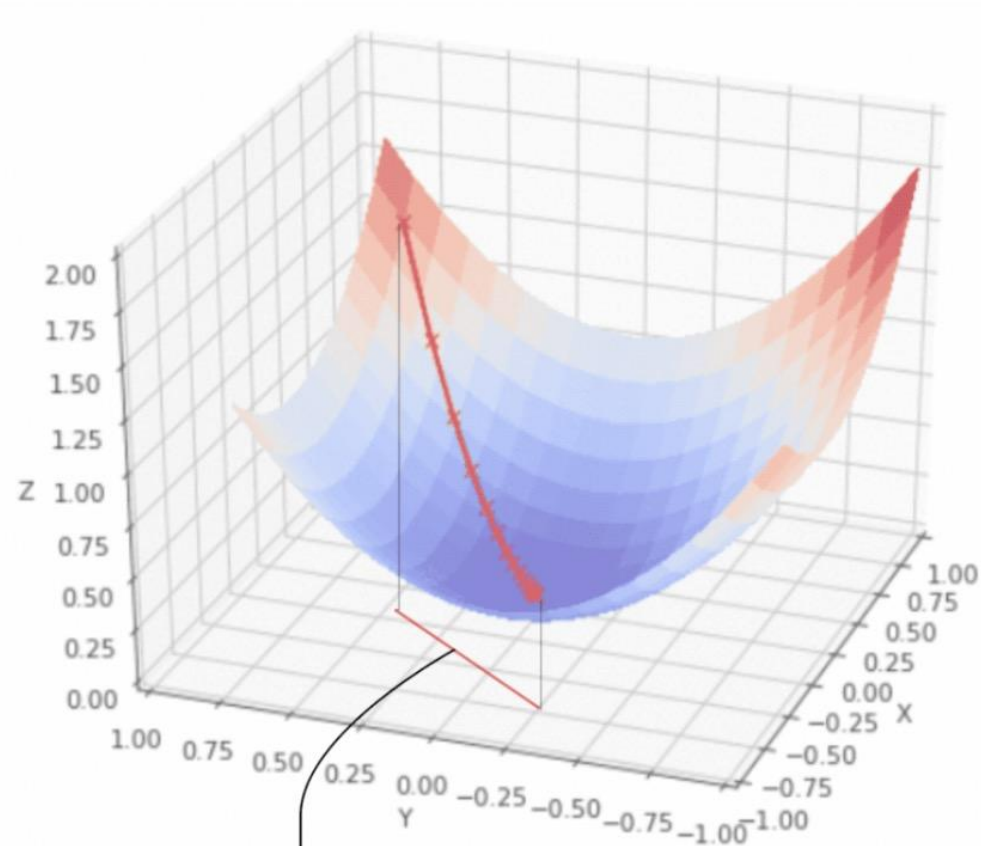
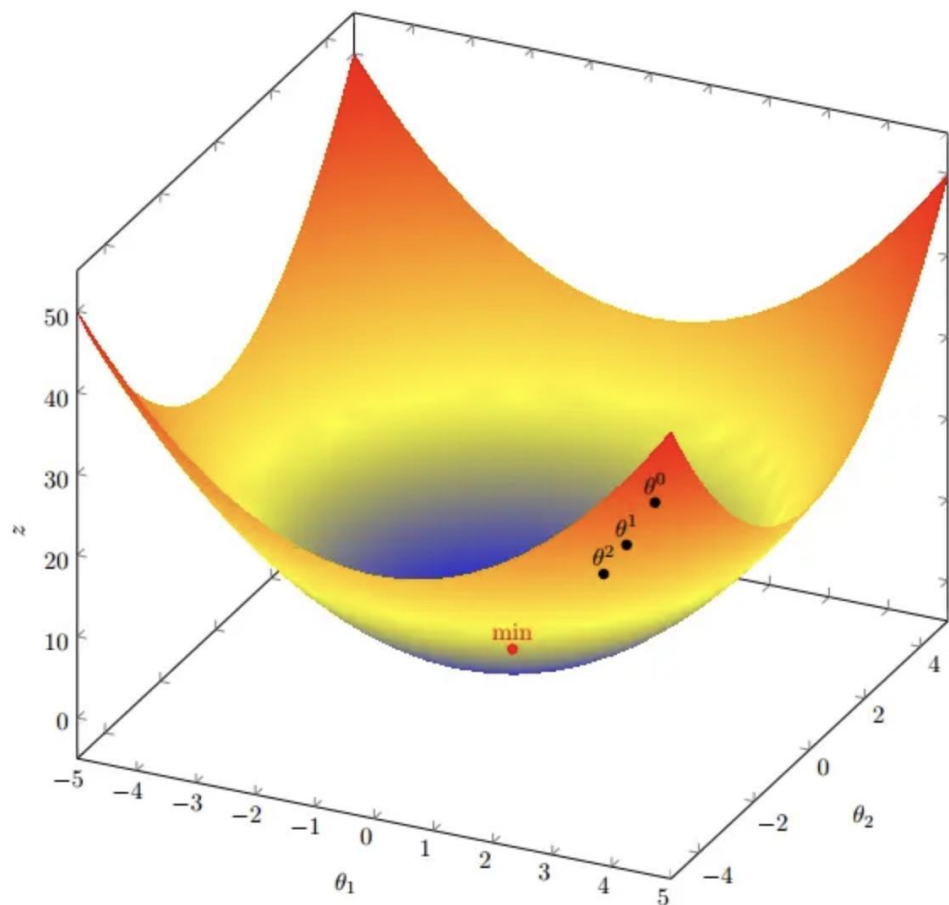
$$\Theta^{50} = (1.1417981541647683e^{-05}, 3.425394462494306e^{-05})$$

$\vdots$

$$\Theta^{100} = (1.6296287810675902e^{-10}, 4.888886343202771e^{-10})$$

4.3 基于梯度的优化方法

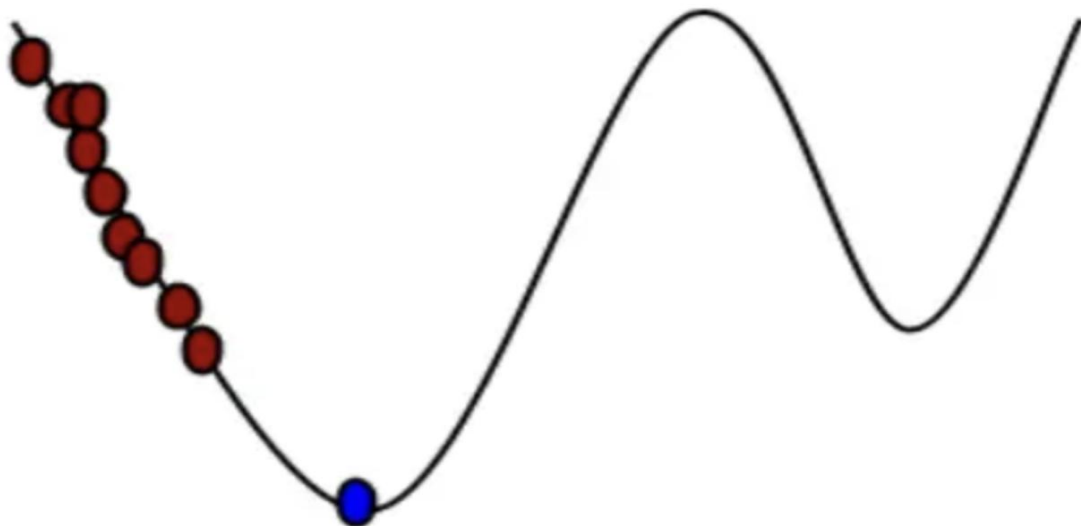
36



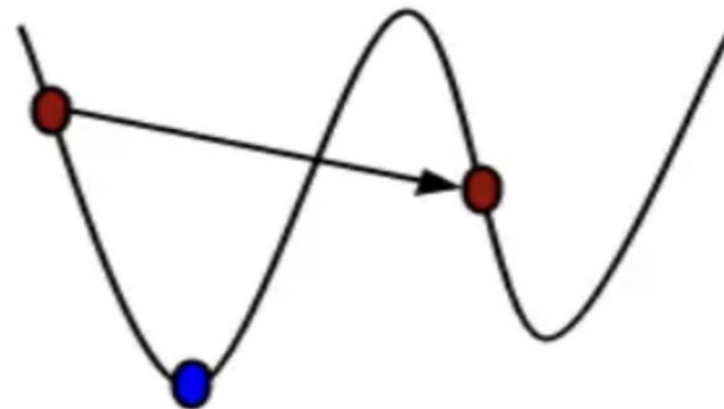
Real Trajectory of G.D.

4.3 基于梯度的优化方法

37



very small learning
rate needs lots of
steps



too big learning rate:
missed the minimum

4.3.1 梯度之上：Jacobian 和 Hessian 矩阵

38

设 $\mathbf{x} \in \mathbb{R}^m$ 为 m 维列向量, $\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_m \end{bmatrix}$, $\mathbf{f}(\mathbf{x}) \in \mathbb{R}^n$ 为 n 维列向量函数,

$\mathbf{f}(\mathbf{x}) = \begin{bmatrix} f_1(\mathbf{x}) \\ f_2(\mathbf{x}) \\ \vdots \\ f_n(\mathbf{x}) \end{bmatrix}$, 则 $\mathbf{f}(\mathbf{x})$ 对 $\mathbf{x}$ 的偏导数, 也称为 $\mathbf{f}(\mathbf{x})$ 对 $\mathbf{x}$ 的雅可比, 常记为 $\mathbf{f}_x(\mathbf{x})$

$$\mathbf{f}_x(\mathbf{x}) = \frac{\partial \mathbf{f}(\mathbf{x})}{\partial \mathbf{x}} = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \frac{\partial f_1}{\partial x_2} & \cdots & \frac{\partial f_1}{\partial x_m} \\ \frac{\partial f_2}{\partial x_1} & \frac{\partial f_2}{\partial x_2} & \cdots & \frac{\partial f_2}{\partial x_m} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial f_n}{\partial x_1} & \frac{\partial f_n}{\partial x_2} & \cdots & \frac{\partial f_n}{\partial x_m} \end{bmatrix} \in \mathbb{R}^{n \times m}$$

结果为 $n \times m$ 的矩阵函数, 即雅可比矩阵的取值依赖于 $\mathbf{x}$ 的取值。

4.3.1 梯度之上：Jacobian 和 Hessian 矩阵

39

考虑函数 $f(x, y) = \begin{bmatrix} x^2 + y^2 \\ x + y \end{bmatrix}$ ，求雅可比矩阵。

4.3.1 梯度之上：Jacobian 和 Hessian 矩阵

40

先求 $f_1(x, y) = x^2 + y^2$ 对 x 和 y 的偏导数：

再求 $f_2(x, y) = x + y$ 对 x 和 y 的偏导数：

$$\frac{\partial f_1}{\partial x} = 2x;$$

$$\frac{\partial f_2}{\partial x} = 1;$$

$$\frac{\partial f_1}{\partial y} = 2y。$$

$$\frac{\partial f_2}{\partial y} = 1。$$

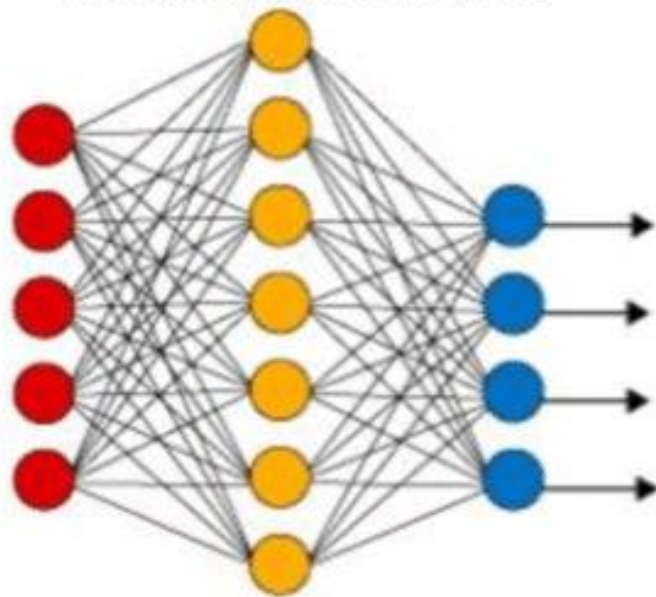
所以雅可比矩阵为：

$$J = \begin{bmatrix} \frac{\partial f_1}{\partial x} & \frac{\partial f_1}{\partial y} \\ \frac{\partial f_2}{\partial x} & \frac{\partial f_2}{\partial y} \end{bmatrix} = \begin{bmatrix} 2x & 2y \\ 1 & 1 \end{bmatrix}。$$

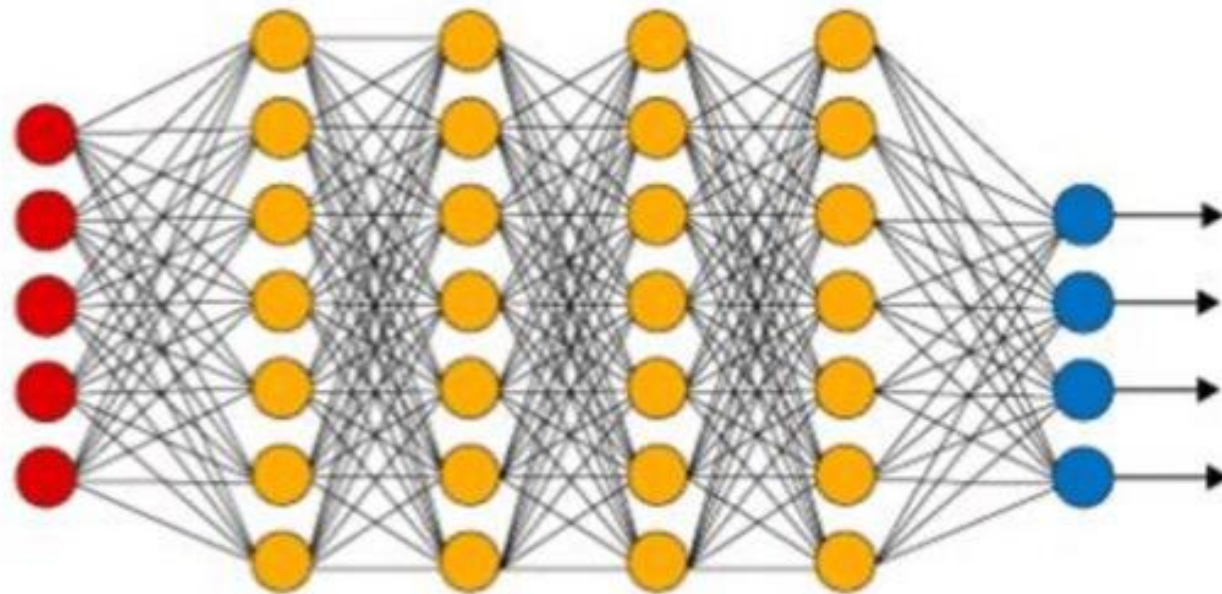
4.3.1 梯度之上：Jacobian 和 Hessian 矩阵

41

Artificial Neural Network



Deep Neural Network



4.3.1 梯度之上：Jacobian 和 Hessian 矩阵

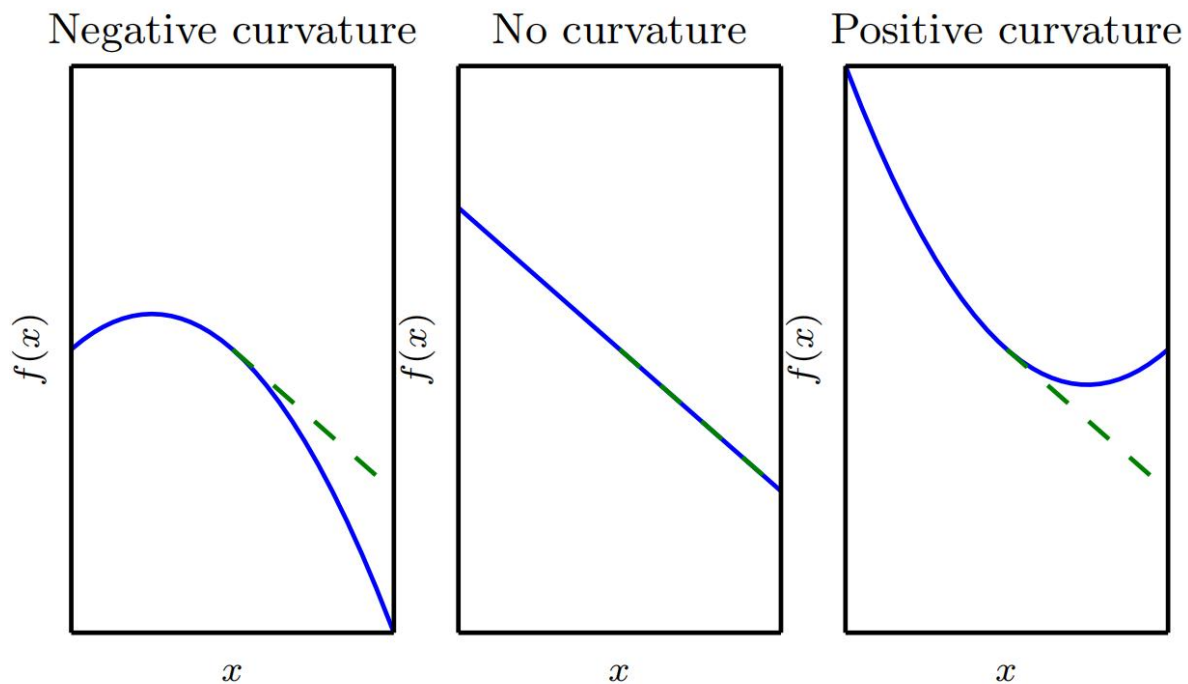
42

有时，我们也对导数的导数感兴趣，即 **二阶导数** (second derivative)。例如，有一个函数 $f : \mathbb{R}^m \rightarrow \mathbb{R}$ ， f 的一阶导数(关于 x_j) 关于 x_i 的导数记为 $\frac{\partial^2}{\partial x_i \partial x_j} f$ 。在一维情况下，我们可以将 $\frac{\partial^2}{\partial x^2} f$ 为 $f''(x)$ 。二阶导数告诉我们，一阶导数将如何随着输入的变化而改变。它表示只基于梯度信息的梯度下降步骤是否会产生如我们预期的那样大的改善，因此它是重要的。我们可以认为，二阶导数是对曲率的衡量。

4.3.1 梯度之上：Jacobian 和 Hessian 矩阵

43

- 如果这样的函数具有零二阶导数，那就没有曲率。也就是一条完全平坦的线，仅用梯度就可以预测它的值。



4.3.1 梯度之上：Jacobian 和 Hessian 矩阵

44

当我们的函数具有多维输入时，二阶导数也有很多。我们可以将这些导数合并成一个矩阵，称为 **Hessian** 矩阵。Hessian 矩阵 $\mathbf{H}(f)(\mathbf{x})$ 定义为

$$\mathbf{H}(f)(\mathbf{x})_{i,j} = \frac{\partial^2}{\partial x_i \partial x_j} f(\mathbf{x}). \quad (4.6)$$

Hessian 等价于梯度的 Jacobian 矩阵。

4.3.1 梯度之上：Jacobian 和 Hessian 矩阵

45

$$f(x, y) = 5x + 8y + xy - x^2 - 2y^2$$

$$\nabla f(x, y) = \begin{bmatrix} \frac{\partial f}{\partial x} \\ \frac{\partial f}{\partial y} \end{bmatrix} = \begin{bmatrix} 5 - 2x + y \\ 8 + x - 4y \end{bmatrix}$$

$$H(x) = \begin{bmatrix} 2 & 1 \\ 1 & -4 \end{bmatrix}$$

$$\frac{\partial^2 f}{\partial x^2} = \frac{\partial(5 - 2x + y)}{\partial x} = -2$$

$$\frac{\partial}{\partial y} \frac{\partial f}{\partial x} = \frac{\partial(5 - 2x + y)}{\partial y} = 1$$

$$\frac{\partial}{\partial x} \frac{\partial f}{\partial y} = \frac{\partial(8 + x - 4y)}{\partial x} = 1$$

$$\frac{\partial^2 f}{\partial y^2} = \frac{\partial(8 + x - 4y)}{\partial y} = -4$$

4.3.1 梯度之上：Jacobian 和 Hessian 矩阵

46

我们可以通过（方向）二阶导数预期一个梯度下降步骤能表现得多么好。我们在当前点 $\mathbf{x}^{(0)}$ 处作函数 $f(\mathbf{x})$ 的近似二阶泰勒级数：

$$f(\mathbf{x}) \approx f(\mathbf{x}^{(0)}) + (\mathbf{x} - \mathbf{x}^{(0)})^\top \mathbf{g} + \frac{1}{2}(\mathbf{x} - \mathbf{x}^{(0)})^\top \mathbf{H}(\mathbf{x} - \mathbf{x}^{(0)}), \quad (4.8)$$

其中 $\mathbf{g}$ 是梯度， $\mathbf{H}$ 是 $\mathbf{x}^{(0)}$ 点的 Hessian。如果我们使用学习率 ϵ ，那么新的点 $\mathbf{x}$ 将会是 $\mathbf{x}^{(0)} - \epsilon \mathbf{g}$ 。代入上述的近似，可得

$$f(\mathbf{x}^{(0)} - \epsilon \mathbf{g}) \approx f(\mathbf{x}^{(0)}) - \epsilon \mathbf{g}^\top \mathbf{g} + \frac{1}{2} \epsilon^2 \mathbf{g}^\top \mathbf{H} \mathbf{g}. \quad (4.9)$$

4.3.1 梯度之上：Jacobian 和 Hessian 矩阵

47

- 牛顿法是一种基于二阶导数的优化算法。其核心思想是利用函数的一阶导数（梯度）和二阶导数（Hessian 矩阵）来确定函数下降的方向和步长，以更快地收敛到函数的最小值。

对于一个目标函数 $f(x)$ ，在某一点 x_k 处，牛顿法通过求解以下方程来确定下一步的更新方向 Δx ：

$$H(f)(x_k)\Delta x = -\nabla f(x_k)$$

其中， $H(f)(x_k)$ 是函数在 x_k 处的 Hessian 矩阵，它表示函数的二阶导数信息； $\nabla f(x_k)$ 是函数在 x_k 处的梯度，表示函数的一阶导数信息。

一旦求解出 Δx ，就可以更新当前的点 $x_{k+1} = x_k + \Delta x$ 。

4.3.1 梯度之上：Jacobian 和 Hessian 矩阵

48

我们可以使用 Hessian 矩阵的信息来指导搜索，以解决这个问题。其中最简单的方法是 **牛顿法**（Newton's method）。牛顿法基于一个二阶泰勒展开来近似 $\mathbf{x}^{(0)}$ 附近的 $f(\mathbf{x})$ ：

$$f(\mathbf{x}) \approx f(\mathbf{x}^{(0)}) + (\mathbf{x} - \mathbf{x}^{(0)})^\top \nabla_{\mathbf{x}} f(\mathbf{x}^{(0)}) + \frac{1}{2}(\mathbf{x} - \mathbf{x}^{(0)})^\top \mathbf{H}(f)(\mathbf{x}^{(0)}) (\mathbf{x} - \mathbf{x}^{(0)}). \quad (4.11)$$

接着通过计算，我们可以得到这个函数的临界点：

$$\mathbf{x}^* = \mathbf{x}^{(0)} - \mathbf{H}(f)(\mathbf{x}^{(0)})^{-1} \nabla_{\mathbf{x}} f(\mathbf{x}^{(0)}). \quad (4.12)$$

4.3.1 梯度之上：Jacobian 和 Hessian 矩阵

49

- 牛顿法
- 在曲率较大的区域，减小步长可以避免过度下降；
- 在曲率较小的区域，增大步长可以加快收敛速度
- 综合考虑函数的一阶导数（梯度）和二阶导数（曲率）信息，可以自适应的调整步长，更高效地朝着函数的最小值点移动

4.3.1 梯度之上：Jacobian 和 Hessian 矩阵

50

$$f(\mathbf{x}^{(0)} - \epsilon \mathbf{g}) \approx f(\mathbf{x}^{(0)}) - \epsilon \mathbf{g}^\top \mathbf{g} + \frac{1}{2} \epsilon^2 \mathbf{g}^\top \mathbf{H} \mathbf{g}. \quad (4.9)$$

最优步长是？

$$\frac{d}{d\epsilon} \left(f(\mathbf{x}^{(0)}) - \epsilon \mathbf{g}^\top \mathbf{g} + \frac{1}{2} \epsilon^2 \mathbf{g}^\top \mathbf{H} \mathbf{g} \right) = -\mathbf{g}^\top \mathbf{g} + \epsilon \mathbf{g}^\top \mathbf{H} \mathbf{g},$$

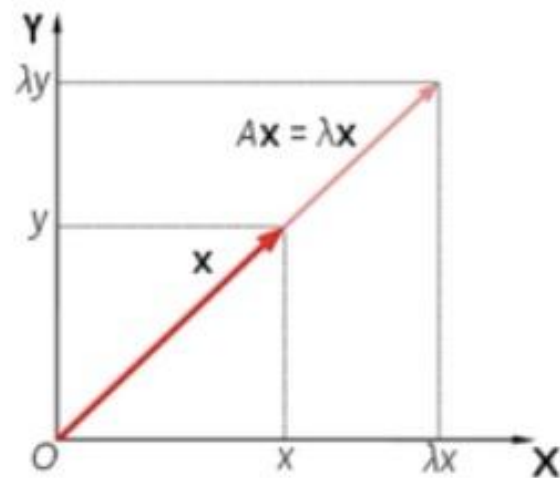
当 $\mathbf{g}^\top \mathbf{H} \mathbf{g}$ 为正时，通过计算可得，使近似泰勒级数下降最多的最优步长为

$$\epsilon^* = \frac{\mathbf{g}^\top \mathbf{g}}{\mathbf{g}^\top \mathbf{H} \mathbf{g}}.$$

4.3.1 梯度之上：Jacobian 和 Hessian 矩阵

51

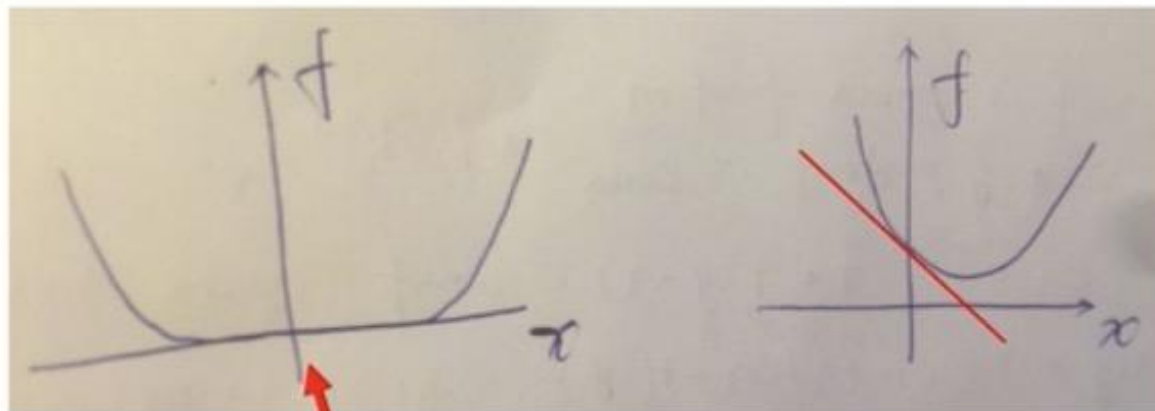
特征值、特征向量



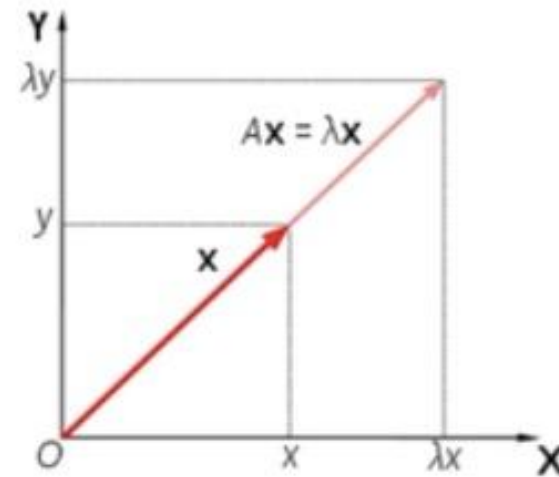
4.3.1 梯度之上：Jacobian 和 Hessian 矩阵

52

特征值、特征向量



$$H_{\lambda} = 0$$



通过考虑 Hessian 的特征值，可以自适应地调整学习率

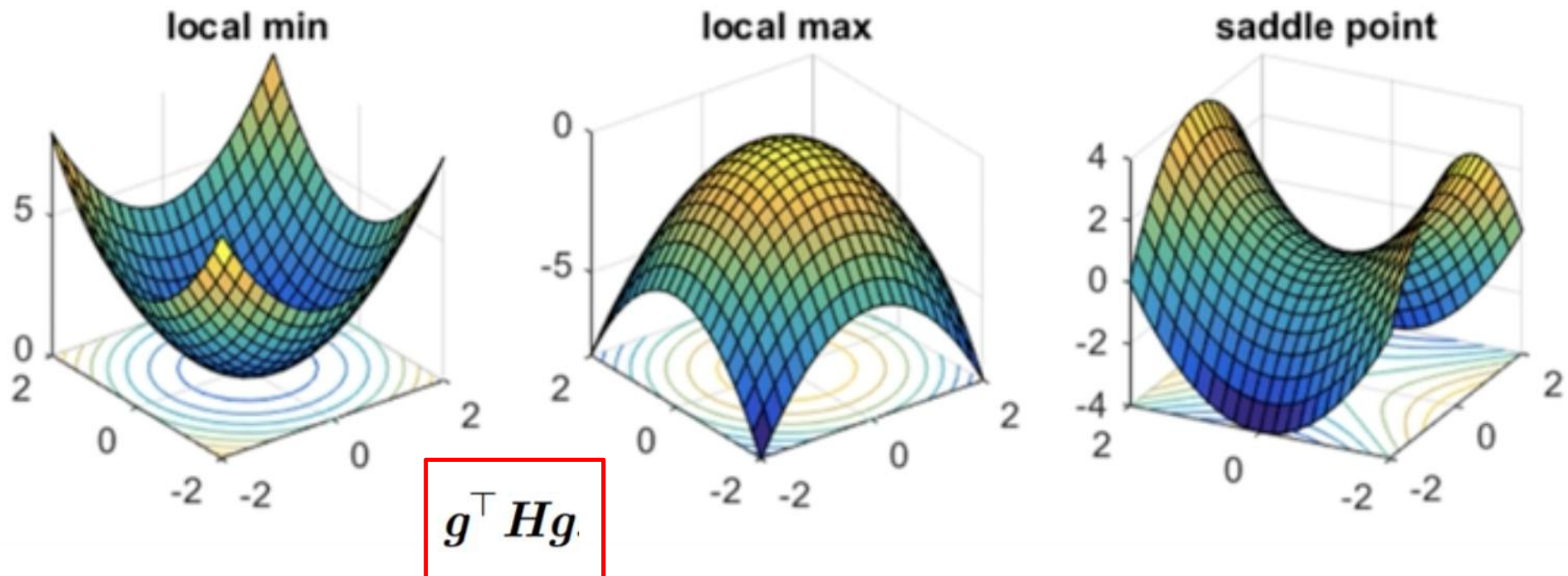
4.3.1 梯度之上：Jacobian 和 Hessian 矩阵

53

二阶导数还可以被用于确定一个临界点是否是局部极大点、局部极小点或鞍点。回想一下，在临界点处 $f'(x) = 0$ 。而 $f''(x) > 0$ 意味着 $f'(x)$ 会随着我们移向右边而增加，移向左边而减小，也就是 $f'(x - \epsilon) < 0$ 和 $f'(x + \epsilon) > 0$ 对足够小的 ϵ 成立。换句话说，当我们移向右边，斜率开始指向右边的上坡，当我们移向左边，斜率开始指向左边的上坡。因此我们得出结论，当 $f'(x) = 0$ 且 $f''(x) > 0$ 时， $\mathbf{x}$ 是一个局部极小点。同样，当 $f'(x) = 0$ 且 $f''(x) < 0$ 时， $\mathbf{x}$ 是一个局部极大点。这就是所谓的二阶导数测试 (second derivative test)。不幸的是，当 $f''(x) = 0$ 时测试是不确定的。在这种情况下， $\mathbf{x}$ 可以是一个鞍点或平坦区域的一部分。

4.3.1 梯度之上：Jacobian 和 Hessian 矩阵

54



如果Hessian矩阵在某点正定，那么函数在该点处是严格的局部极小值；
如果Hessian矩阵在某点负定，那么函数在该点处是严格的局部极大值；
如果Hessian矩阵在某点不定，那么函数在该点处既不是极小值也不是极大值。

4.3.1 梯度之上：Jacobian 和 Hessian 矩阵

55

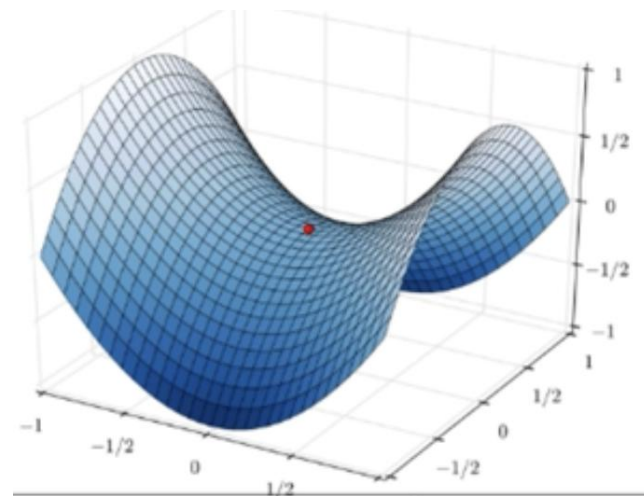
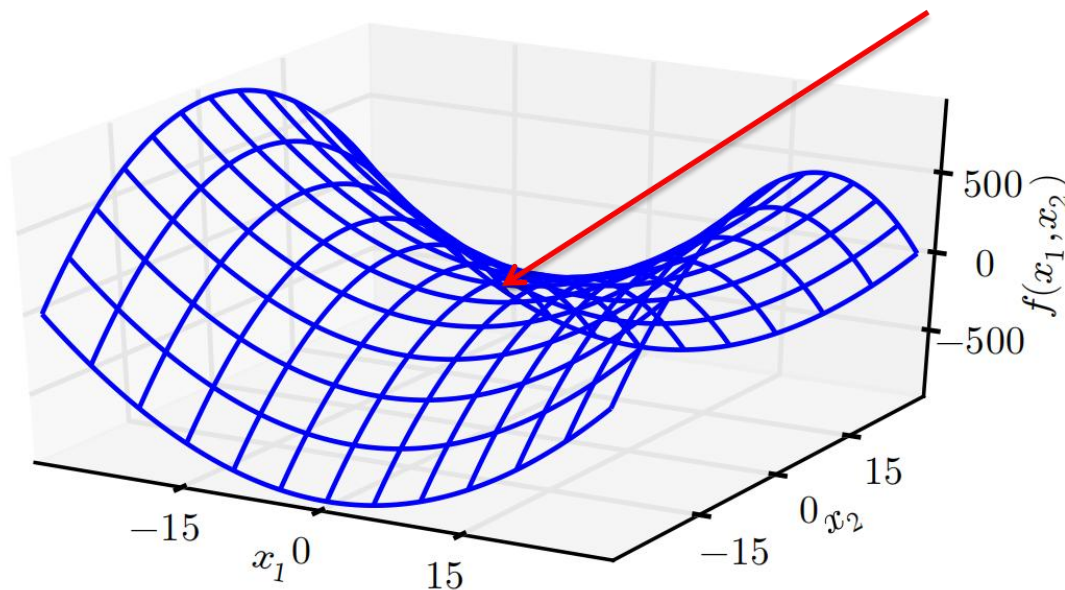


图 4.5: 既有正曲率又有负曲率的鞍点。示例中的函数是 $f(\mathbf{x}) = x_1^2 - x_2^2$ 。函数沿 x_1 轴向上弯曲。 x_1 轴是 Hessian 的一个特征向量，并且具有正特征值。函数沿 x_2 轴向下弯曲。该方向对应于 Hessian 负特征值的特征向量。名称“鞍点”源自该处函数的鞍状形状。这是具有鞍点函数的典型示例。维度多于一个时，鞍点不一定要具有 0 特征值：仅需要同时具有正特征值和负特征值。我们可以想象这样一个鞍点（具有正负特征值）在一个横截面内是局部极大点，而在另一个横截面内是局部极小点。

4.3.1 梯度之上：Jacobian 和 Hessian 矩阵

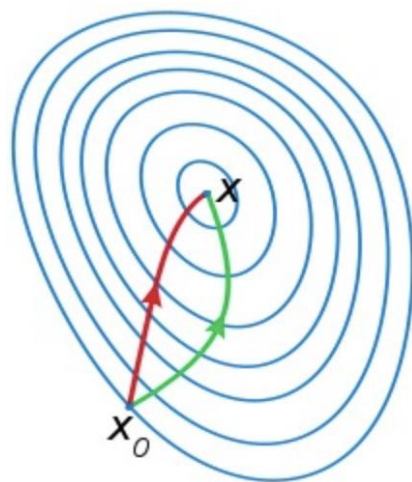
56

牛顿法 (Newton's Method)，也称为牛顿-拉弗森法 (Newton-Raphson Method)，是一种用于求解方程根或函数的最小值的迭代优化算法。它利用函数的二阶导数信息 (Hessian矩阵) 来逼近函数的局部性质，能够更快地收敛到最优解。

4.3.1 梯度之上：Jacobian 和 Hessian 矩阵

57

- 仅使用梯度信息的优化算法被称为 一阶优化算法 (first-order optimization algorithms), 如梯度下降。
- 使用 Hessian 矩阵的优化算法被称为 二阶最优化算法(second-order optimization algorithms)(Nocedal and Wright, 2006), 如牛顿法。



红色曲线是利用牛顿法迭代求解
绿色曲线是利用梯度下降法求解